Ownership

Lukáš Hozda

NPRG082 · winter 2026/27

Values and resources

- Ownership: responsibility for an owned value
- Moves, copies, and explicit duplication
- Destruction and resource lifetime
- Borrowing: access without ownership transfer

Resource lifetime

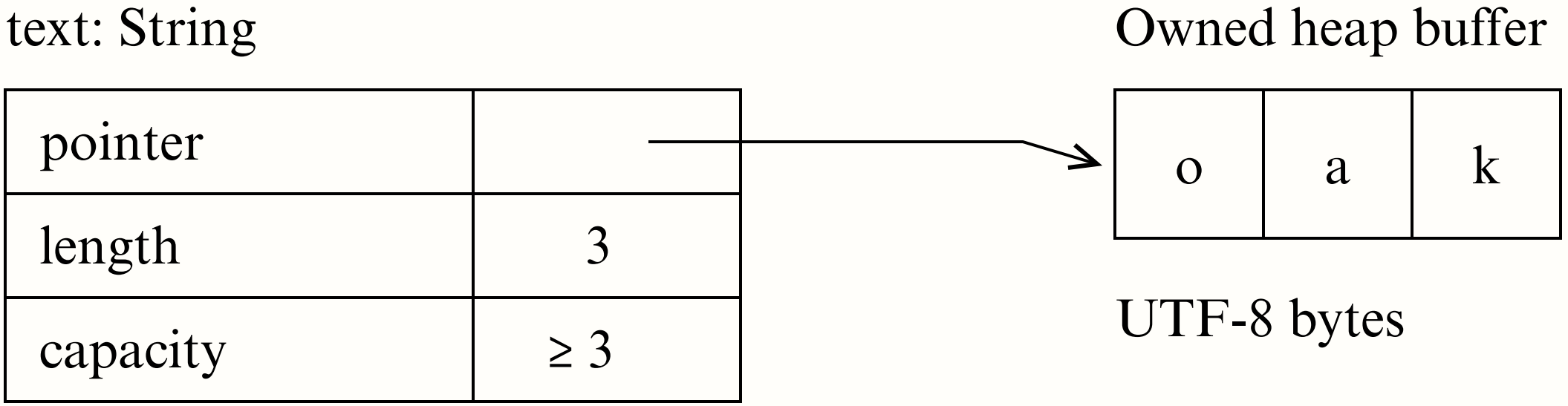
- Allocated memory, open files, held locks
- Access after release: invalid resource
- Two independent releases: double cleanup
- Ownership and borrowing: compile-time rules for valid access

Values and bindings

```
fn main() {
    let text = String::from("oak");
    println!("{}", text.len());
}
```

- `text`: a local binding containing a `String`
- The `String` owns a buffer of UTF-8 bytes

Owned storage



- Length: bytes in use; capacity: allocated space
- Conceptual fields, not a specified field order

Storage and ownership

- Local values: usually stored in the stack frame
- `String` and `Vec<T>`: owned heap buffers
- Ownership rules apply to heap-free values too
- Storage placement and machine copies: compiler decisions

Scope

```
fn main() {
    {
        let text = String::from("oak");
        println!("{}", text.len());
    }
}
```

- The local value is dropped at the end of its scope
- Dropping this `String` releases its buffer

Ownership transfer

```
fn main() {
    let first = String::from("oak");
    let second = first;
    println!("{second}");
}
```

- Assignment moves a non-`Copy` value
- `second` now owns the string

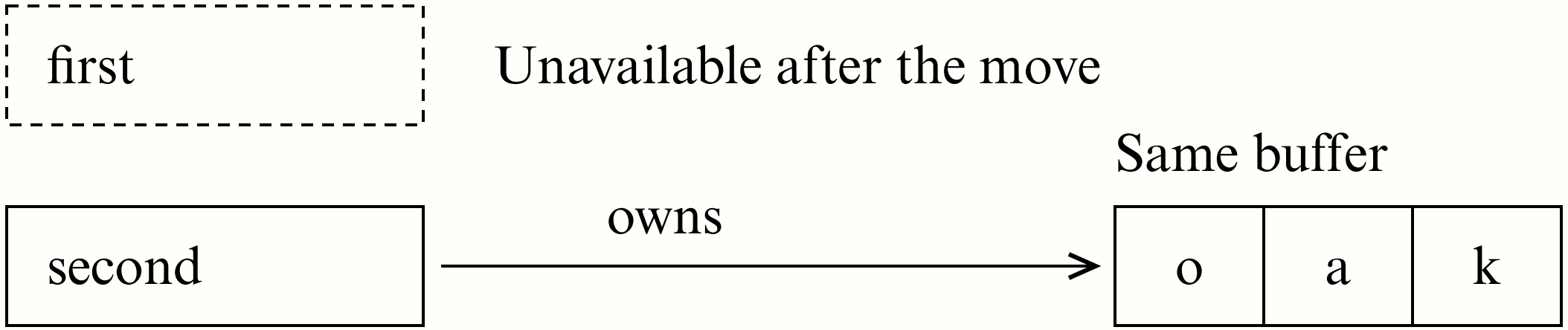
Use after move

```
fn main() {
    let first = String::from("oak");
    let second = first;
    println!("{second}");
    println!("{first}");
}
```

- E0382: use of a moved value

String move

After let second = first;



- Ownership transfer without duplicating the buffer
- The old binding has no value available for use or drop

Moves without allocation

```
struct Token;
fn main() {
    let first = Token;
    let _second = first;
    let _third = first;
}
```

- Even a zero-sized type is non-`Copy` by default

Reinitialization

```
fn main() {
    let mut first = String::from("oak");
    let second = first;
    first = String::from("elm");
    println!("{first} {second}");
}
```

- A moved binding can receive a new value

Owned parameters

```
fn byte_len(text: String) -> usize {
    text.len()
}
fn main() {
    let name = String::from("oak");
    println!("{}", byte_len(name));
}
```

- The argument moves into the parameter; the parameter is dropped on return

Return values

```
fn name() -> String {
    let text = String::from("oak");
    text
}
fn main() {
    let result = name();
    println!("{result}");
}
```

- Returning the value transfers ownership to the caller

Implicit copies

```
fn main() {
    let first = 7;
    let mut second = first;
    second += 1;
    println!("{first} {second}");
}
```

- `i32` implements `Copy`: both bindings have usable values

Copy as a type property

- Implicit, bitwise duplication in value contexts
- Integers, floats, `bool`, `char`; tuples and arrays of `Copy` elements
- User-defined types: explicit implementation or derivation
- Type size and stack placement do not decide whether a type is `Copy`

Derived Copy

```
#[derive(Clone, Copy)]
struct Pair(i32, i32);
fn main() {
    let first = Pair(3, 4);
    let second = first;
    println!("{} {}", first.0, second.1);
}
```

- Every field must be `Copy`; a type with `Drop` cannot be `Copy`

Explicit duplication

```
fn main() {
    let first = String::from("oak");
    let mut second = first.clone();
    second.push('s');
    println!("{first} {second}");
}
```

- `String::clone`: a separate owned buffer with the same contents

Move, copy, clone

- **Move:** ownership transfer; source value unavailable
- **Copy:** implicit duplication; source value usable
- **Clone:** explicit, type-defined duplication
- A clone may allocate; shared-pointer clones can share a resource

Partial moves

```
fn main() {
    let pair = (String::from("L"), String::from("R"));
    let left = pair.0;
    println!("{left} {}", pair.1);
}
```

- Moving one field leaves the other field available

Whole value after a partial move

```
fn main() {
    let pair = (String::from("L"), String::from("R"));
    let _left = pair.0;
    println!("{pair:?}");
}
```

- E0382: the whole tuple is no longer available

Conditional moves

```
fn main() {
    let text = String::from("oak");
    if text.len() > 2 {
        let _taken = text;
    }
    println!("{text}");
}
```

- A value must be available on every path reaching its use

Destruction

- Scope exit: drop the initialized local values
- Reverse declaration order for local bindings
- Recursive cleanup of owned fields
- `Drop`: additional type-specific cleanup

Observable drop order

```
struct Trace(u8);
impl Drop for Trace {
    fn drop(&mut self) { print!("{} ", self.0); }
}
fn main() {
    let _outer = Trace(1);
    { let _inner = Trace(2); }
    print!("end ");
}
```

- Inner scope first, then the remaining outer local

Early release

```
fn main() {
    let text = String::from("oak");
    drop(text);
    println!("{text}");
}
```

- `std::mem::drop` takes its argument by value
- E0382: ownership was transferred into `drop`

Replacement

```
struct Trace(u8);
impl Drop for Trace {
    fn drop(&mut self) { print!("{} ", self.0); }
}
fn main() {
    let mut value = Trace(1);
    value = Trace(2);
    print!("body ");
    drop(value);
}
```

- Assignment drops the old value before storing the new one

Resource management

- Cleanup follows ownership, including ordinary early returns
- Unwinding: cleanup of initialized values on the unwound path
- Process exit and abort can skip destructors
- Memory safety does not imply freedom from resource leaks

Borrowed access

```
fn main() {
    let value = 7;
    let view = &value;
    println!("{} {value}", *view);
}
```

- `&value`: shared reference; `*view`: access to the referent
- Borrowing leaves ownership with the original owner

Shared references

```
fn main() {
    let text = String::from("oak");
    let first = &text;
    let second = first;
    println!("{first} {second}");
}
```

- `&T` is `Copy`, even when `T` is not
- Copying a reference does not duplicate the owned value

Mutable references

```
fn main() {
    let mut value = 7;
    let view = &mut value;
    *view += 1;
    println!("{value}");
}
```

- Exclusive access through `&mut T`; `&mut T` is not `Copy`
- The reference binding itself need not be `mut`

Mutable bindings

```
fn main() {
    let first = 1;
    let second = 2;
    let mut view = &first;
    print!("{view} ");
    view = &second;
    println!("{view}");
}
```

- `mut view`: reassignment of the binding, not mutation through `&T`

Conflicting access

```
fn main() {
    let mut text = String::from("oak");
    let view = &text;
    text.push('s');
    println!("{view}");
}
```

- E0502: shared access overlaps the mutable access required by `push`

Last use of a borrow

```
fn main() {
    let mut text = String::from("oak");
    let view = &text;
    print!("{view} ");
    text.push('s');
    println!("{text}");
}
```

- The borrow can end after its last use, before the binding's scope ends

Exclusive access

```
fn main() {
    let mut value = 7;
    let first = &mut value;
    let second = &mut value;
    println!("{first} {second}");
}
```

- E0499: overlapping mutable borrows of the same value

Reborrowing

```
fn main() {
    let mut value = 7;
    let outer = &mut value;
    {
        let inner = &mut *outer;
        *inner += 1;
    }
    *outer += 1;
    println!("{value}");
}
```

- Temporary access through a shorter borrow

Reference lifetime

```
fn main() {
    let view;
    {
        let text = String::from("oak");
        view = &text;
    }
    println!("{view}");
}
```

- E0597: the referent must exist for every use of the reference

Owned strings and string slices

- `String`: growable, owned UTF-8 buffer
- `&str`: borrowed view of UTF-8 bytes, with a length
- String literals: borrowed text with static storage
- Borrowed views do not allocate or extend the owner's lifetime

String slices

```
fn main() {
    let text = String::from("red green");
    let prefix = &text[..3];
    println!("{prefix} {text}");
}
```

- Slice indices are byte offsets
- String slice boundaries must be valid UTF-8 boundaries

Sequence slices

```
fn main() {
    let values = vec![10, 20, 30];
    let tail = &values[1..];
    println!("{tail:?}");
}
```

- `&[T]`: borrowed contiguous elements, with a length
- One owned buffer, accessed through a smaller view

Vector growth

```
fn main() {
    let mut values = Vec::with_capacity(4);
    values.push(10);
    let first = &values[0];
    values.push(20);
    println!("{first}");
}
```

- `push` requires `&mut Vec<T>`, even with spare capacity
- Its signature requires exclusive access, rather than a proof of reallocation

Borrowed parameters

```
fn byte_len(text: &str) -> usize { text.len() }
fn main() {
    let owned = String::from("oak");
    let length = byte_len(&owned);
    println!("{length} {}", byte_len("elm"));
}
```

- A `&str` parameter accepts a borrowed `String` or a string literal

Borrowed results

```
fn trimmed(text: &str) -> &str { text.trim() }
fn main() {
    let owned = String::from(" oak ");
    let view = trimmed(&owned);
    println!("{view}");
}
```

- The result borrows the input; no new string allocation
- With one input lifetime, the output lifetime is inferred from it

Ownership in signatures

```
fn inspect(text: &str) { /* shared access */ }
fn change(text: &mut String) { /* exclusive access */ }
fn consume(text: String) { /* owned input */ }
fn create() -> String { String::new() }
```

- Shared access, exclusive access, or ownership transfer
- The signature expresses the caller's obligations

Ownership choices

- Borrow for temporary access
- Transfer ownership when the recipient needs the value
- Clone when a second owned value is needed
- Return an owned value for newly created data

Ownership model

- Each owned value: one owner at a time
- Move: transfer; `Copy`: implicit duplicate; `Clone`: explicit duplicate
- Destruction: cleanup of the values still owned
- Borrowing: bounded shared or exclusive access

References

- *The Rust Book: Understanding Ownership*
- *The Rust Reference: Destructors*
- *`Copy` and `Clone`*
- *`String` representation*
- *`Vec` guarantees*